

Tema 02

Datos, expresiones y algoritmos

Programación en Lenguaje C

Ingeniería de Sistemas y Automática - Universidad de Oviedo

Tipos de datos básicos

Los datos pueden ser de varios tipos:

- **int**: Para almacenar y realizar operaciones con valores enteros.
- **float**: Para almacenar y realizar operaciones con valores reales.
- **char**: Para almacenar y realizar operaciones con texto.

El programa puede utilizar datos de dos procedencias:

- **Variables**: Valores que pueden cambiar a lo largo de la ejecución.

➤ **Ejs:**

```
int n_alumnos;           // Variable tipo entero, sin inicializar
float nota_media = 7.25; // Variable tipo real, valor inicial 7.25
```

- **Constantes**: Valores fijos que no cambian a lo largo del programa.

➤ **Ejs:** 7 (días de la semana), 3.1416 (π), "Introduzca x1:" (texto), etc.

➤ **Preferible declararlas con #define al principio del código:**

```
#define PI 3.1416
```

y usar después mediante el nombre:

```
float z = 2 * cos(PI / 4);
```

Sentencias y Expresiones

Sentencia básica: Asignación.

`variable = expresión;`

Expresión: Resultado de una operación matemática con operandos y operadores:

- Operadores aritméticos: +, -, *, /, paréntesis (), etc.
- Operandos: Variables, constantes o funciones matemáticas (cos, sin, sqrt, etc.).

```
#include <math.h>
// Ejemplo de sentencia con funciones matemáticas
z = 3 * y + (2 * cos(PI / 4));
```

Evaluación de Expresiones

División secuencial de operaciones según la prioridad de los operadores

Sentencia C:

```
z = 3 * y + (2 * cos(PI / 4));
```

Orden de precedencia:

1. Paréntesis () y funciones
2. Multiplicación * y División /
3. Suma + y Asignación =

Secuencia de evaluación por la ALU (Resultados parciales):

Paso 1 (Paréntesis)

1. $PI / 4 \rightarrow r1$

Paso 2 (Función)

2. $\cos(r1) \rightarrow r2$

Paso 3 (Producto interno)

3. $2 * r2 \rightarrow r3$

Paso 4 (Producto izqda.)

4. $3 * y \rightarrow r4$

Paso 5 (Suma final)

5. $r4 + r3 \rightarrow r5$

Paso 6 (Escritura en RAM)

6. $z = r5$

Operaciones parciales y tipos

Cada operación parcial utiliza los tipos de sus operandos y devuelve un resultado de ese mismo tipo:

- `op1 (int) OP opt2 (int) → int`
- `op1 (float) OP opt2 (float) → float`
- `op1 (int) OP opt2 (float) → float`
- `op1 (float) OP opt2 (int) → float`

```
int x = 4, y = 8;
float z;

z = x / y; // ¡ATENCIÓN! Operación entera: 4 / 8 resulta en 0.
           // z toma el valor 0.0 (float)
```

Operador Molde (Cast)

Para forzar una conversión explícita de tipo sin cambiar la naturaleza de la variable original:

(tipo) operando

```
int profes = 4, alumnos = 8;
float ratio;

// Conversión explícita a float de los operandos
ratio = (float)profes / (float)alumnos;

// Ejecución: 4.0f / 8.0f = 0.5f -> ratio pasa a valer 0.5
```

Sentencias de Control

```
// Bloque
{
    sentencia1;
    sentencia2;
}

// Alternativa
if (condicion)
    sentenciaSiTRUE;
else
    sentenciaSiFALSE;

// Bucles
while (condicion)
    sentenciaARepetir;

for (init; condicion; paso)
    sentenciaARepetir;
```

Bloque { }

Agrupar múltiples sentencias como una sola.

Alternativa if / else

Ejecución condicional (else es opcional).

Bucles while / for

Repetir instrucciones mientras la condición sea verdadera.

Evaluación de Condiciones

En C, las condiciones se evalúan como enteros:

- FALSO: Valor igual a 0.
- VERDADERO: Cualquier valor distinto de 0.

Operadores Relacionales: ==, !=, >, >=, <, <=.

Operadores Lógicos: && (AND), || (OR), ! (NOT).

```
// Ejemplo: y = min(|x|, 4)
if ((x > 4) || (x < -4)) {
    y = 4;
} else {
    if (x >= 0)
        y = x;
    else
        y = -x;
}
```

Diseño de Algoritmos

Un algoritmo traslada el pensamiento lógico a una secuencia ejecutable de instrucciones.

Ejemplo: Calcular el factorial: $fact = n!$

Expresión matemática abstracta: $fact = n \cdot (n-1) \cdot (n-2) \dots \cdot 1$

Secuencia algorítmica:

1. Inicializar **fact** = 1.
2. Repetir desde **i** = 1 hasta **n**:
 fact = **fact** * **i**.
3. Al finalizar el bucle: **fact** contiene **n!**.

Traducción del Algoritmo a C

```
#include <stdio.h>

int main(void) {
    int n, i, fact;

    printf("Introduzca n: ");
    scanf("%d", &n);

    fact = 1;
    for (i = 1; i <= n; i++) {
        fact = fact * i;
    }

    printf("%d! = %d\n", n, fact);
    return 0;
}
```

Depuración de Programas

Un programa sin errores de compilación sintácticos no garantiza un comportamiento correcto.

Causas habituales de fallos:

- Algoritmo mal concebido o mala traducción al lenguaje.
- Uso incorrecto de tipos de datos o modificación indebida de variables.
- Casos límite u ocasionales no contemplados.

Depuración: Proceso de prueba, localización y corrección de errores en el código.

Tipos de datos

char, float, int son los tipos de datos más comunes en C.

En ocasiones, hay que seleccionar otros tipos para ajustarse al rango/resolución de los valores a manejar:

Tipos de Datos Enteros

Selección del tipo según la resolución, rango y memoria requerida

El estándar C solo garantiza un **tamaño mínimo relativo** ($\text{char} \leq \text{short} \leq \text{int} \leq \text{long}$). El compilador concreto asigna los bits reales según la arquitectura física del procesador.

Tipo	Nº bits mín. (Estándar C)	Nº bits (VC++)	Rango Unsigned (unsigned)	Rango Signed (por defecto)
char	8 bits	8 bits	0 a 255	-128 a 127
short	$\geq \text{char}$ (≥ 16 bits)	16 bits	0 a 65.535	-32.768 a 32.767
int	$> \text{char}$ / $\geq \text{short}$	32 bits	0 a 4.294.967.295	-2.147.483.648 a 2.147.483.647
long	$> \text{short}$ / $\geq \text{int}$	32 bits	0 a 4.294.967.295	-2.147.483.648 a 2.147.483.647

Regla del modificador `unsigned`: Elimina la posibilidad de almacenar valores negativos, duplicando el límite superior positivo al aprovechar el bit de signo.

Tipos de Datos Reales (Coma Flotante)

Tipo	Nº bits mín.	Nº bits VC++	Capacidad de representación	
			Origen del Máximo (Base 2 → Base 10)	Resolución Decimal
float	32	32 (4B)	$2^{128} \rightarrow 3.4 \times 10^{38}$	≥ 6-7 dígitos decimales
double	≥ float	64 (8B)	$2^{1024} \rightarrow 1.8 \times 10^{308}$	≥ 15-17 dígitos decimales
long double	> float ≥ double	64 (8B)	<i>Idéntico a double</i>	

¿De dónde sale el valor máximo?

- El hardware guarda la escala en potencias de base 2. El límite de bits de `float` permite escalar hasta 2^{128} , que equivale a 3.4×10^{38} en notación decimal.
- En `double`, al tener el doble de memoria, el límite salta a 2^{1024} , equivalente a 1.8×10^{308} .

Estructuras de Datos

Agrupación de datos heterogéneos bajo una misma entidad

- Es un tipo de dato compuesto definido por el programador que permite agrupar múltiples variables de **distintos tipos** (llamados *campos*) bajo un único nombre.

Representación Conceptual de un Registro / Estructura en Memoria

Estructura: Medición

id
(int)

temp
(float)

estado
(char)



Acceso a Miembros:

Se accede a cada componente individual mediante su **nombre de campo** (usando el operador punto .).

Estructuras en C (struct)

Permiten agrupar variables de distintos tipos bajo un mismo nombre:

```
// Definición de la estructura
struct color {
    float red, green, blue;
};

struct circulo {
    float centrox, centroy, radio;
    struct color colorBorde, colorFondo;
    int anchoBorde, dibujarFondo;
};

// Uso e inicialización
struct circulo sol;
sol.colorFondo.red = 1.0;
sol.colorFondo.green = 1.0;
sol.colorFondo.blue = 0.0;
sol.dibujarFondo = 1;
```

Ejercicios Propuestos

1. Cálculo de raíces reales o complejas de $a \cdot x^2 + b \cdot x + c = 0$

$$x = \frac{-b \pm \sqrt{b^2 - 4 \cdot a \cdot c}}{2 \cdot a}$$

2. Realizar un programa que calcule el nº de equipos titulares diferentes que puede realizar un entrenador, según el nº de jugadores convocados y el nº de jugadores que se alinean

$$C_{n,m} = \frac{n!}{m! \cdot (n - m)!}$$

3. Realizar un programa pida un nº entre 0 y 255, y escriba el valor binario equivalente (8 bits).

4. Comprobar que las dos aproximaciones siguientes se acercan al número e cuando $n \rightarrow \infty$:

$$e = \sum_{n=0}^{\infty} \frac{1}{n!} \quad \text{vs} \quad \lim_{n \rightarrow \infty} \left(1 + \frac{1}{n} \right)^n$$

Resumen del Tema

1. Elección adecuada de tipos según rango y precisión.
2. Atención al tipo de dato en divisiones (usar casting si se requiere decimal).
3. Estructuración de algoritmos previa a la codificación.
4. Uso correcto de la indentación y bloques en sentencias de control.
5. Importancia de la fase de depuración para garantizar un correcto funcionamiento.